

Initial Technology Exploration Summary

The project began by evaluating multiple 5G signal processing approaches, ultimately leading to the breakthrough discovery of OpenAirInterface5G as the optimal foundation.

Python Library Assessment

Sionna and toolkit5G proved excellent for simulation but inadequate for real-time RF processing. Daniel Estévez's implementation provided valuable insights but contained hardcoded parameters (PCI, SCS, SFN) that limited general-purpose application. MATLAB's 5G Toolbox offered comprehensive capabilities but was financially impractical for open-source development.

OpenAirInterface5G Breakthrough

OAI emerged as the game-changer, providing a production-ready, standards-compliant 5G stack with advanced features like CSI analysis and blind synchronization. The RFSimulator enabled hardware-free development, while the Replay Node facilitated testing with captured IQ samples, though format compatibility challenges persisted.

Implementation Architecture

Function mapping revealed OAI's logical organization mirroring the 5G protocol stack, enabling systematic extraction of critical functions like `nr_initial_sync()` and `polar_decoder()`. The six-stage GNU Radio workflow balanced native blocks (FFT, correlation, QPSK demodulation) with custom 5G-specific implementations, maximizing performance while minimizing development complexity.

Practical Validation

PSS implementation using `pss_nr.py` successfully integrated OAI's proven algorithms into GNU Radio Python blocks, demonstrating effective PSS correlation and NID2 detection. Despite initial replay node timing alignment issues, the approach validated the feasibility of bridging OAI's mature C implementation with GNU Radio's flexible flow-graph architecture for comprehensive 5G cell scanning capabilities.

Initial Technology Exploration

Python-Based DSP Libraries Investigation

The exploration began by examining several Python-based libraries for 5G signal processing capabilities:

[Sionna \(NVIDIA\)](#) emerged as a sophisticated GPU-accelerated library designed primarily for AI/ML research in wireless communications. While technically impressive with its differentiable architecture and TensorFlow integration, Sionna's focus on simulation and 6G research made it less suitable for real-world passive 5G cell scanning applications.

[toolkit5G](#) provided a comprehensive Python-based approach with 3GPP standards-compliant downlink and uplink chains. The library offered excellent modularity and speed optimization, but like Sionna, was primarily designed for simulation rather than real-time signal processing of captured RF data.

[py3gpp](#) represented additional Python alternatives, but analysis revealed similar limitations in transitioning from simulation to real-world signal processing applications.

Daniel Estévez's Implementation Analysis

A critical evaluation of Daniel Estévez's 5G NR PBCH decoding work revealed both strengths and limitations. His Jupyter notebook implementation demonstrated effective PBCH decoding approaches and provided valuable insights into synchronization techniques. However, the implementation contained several hardcoded values specific to particular signal recordings, including:

- Physical Cell ID (PCI) - Hardcoded to specific values rather than supporting the full range (0-1007)
- Subcarrier Spacing (SCS) - Fixed to 15 kHz instead of supporting multiple 5G NR numerologies
- System Frame Number (SFN) - Using fixed values rather than continuous incrementing
- CORESET Configuration - Hardcoded parameters instead of deriving from system information

These limitations meant extensive adaptation would be required for a general-purpose scanner implementation. These hardcoded values make educational implementations more predictable and easier to understand, but they represent parameters that would be dynamically configured, negotiated, or adapted in real 5G NR deployments based on network conditions, traffic patterns, and device capabilities

https://github.com/daniestevez/jupyter_notebooks/blob/master/5G/5G%20NR%20downlink.ipynb

<https://destevez.net/2024/12/5g-nr-pbch/>

MATLAB Consideration

MATLAB's 5G Toolbox offered comprehensive capabilities for cell search algorithms, MIB recovery, and SIB1 decoding. However, the lack of available subscription made this financially impractical for an open-source project, despite its technical advantages.

OpenAirInterface5G: The Game Changer

Complete 5G Software Stack Architecture

OpenAirInterface5G (OAI) emerged as the breakthrough solution, offering a complete open-source 5G software stack that addressed all the limitations encountered with previous approaches. The OAI 5G RAN project provides several compelling advantages:

<https://gitlab.eurecom.fr/oai/openairinterface5g>

Production-Ready Implementation

The gNB procedures PHY and MAC layers are production-ready and tested with commercial UE devices for standard compliance. This production readiness includes advanced features such as:

- CSI (Channel State Information) analysis for comprehensive channel quality assessment
- Blind synchronization capabilities enabling robust signal acquisition
- Multi-antenna MIMO support for advanced spatial processing techniques

https://gitlab.eurecom.fr/oai/openairinterface5g/-/blob/develop/doc/FEATURE_SET.md#openairinterface-5g-nr-ue-feature-set

gNB and UE Architecture Framework

The system implements a complete gNB and UE architecture supporting both deployment scenarios :

- gNB (gNodeB): The 5G base station handling radio access network functions
- UE (User Equipment): The 5G device/terminal connecting to the network
- NSA (Non-Standalone) and SA (Standalone) configurations providing deployment flexibility

Hardware vs. Software: The RFSimulator Solution

OAI typically operates with actual RF hardware like USRP devices, but for development and testing without physical hardware, OAI provides the innovative RFSimulator. Key capabilities include:

- Software RF replacement: Replaces RF board with software simulation
- TCP/IP communication: Uses network protocols instead of actual RF interface
- No real-time constraints: Allows faster-than-real-time operation for development
- MIMO support: Enables advanced antenna configurations in simulation
- Development flexibility: Enables functional testing without RF hardware

The RFSimulator is invoked using the `--rfsim` command-line option, making it ideal for development and testing scenarios.

<https://gitlab.eurecom.fr/oai/openairinterface5g/-/tree/develop/radio/rfsimulator>

The Replay Node: Bridging Captured Data and Live Processing

Understanding Replay Node Architecture

The Replay Node is a critical software component within OpenAirInterface5G designed to feed pre-recorded RF data (IQ samples) into the signal processing chain. Instead of relying on live SDR sources, the replay node reads from stored datasets and outputs them as a continuous sample stream, enabling consistent testing and validation using identical waveform captures.

Supported File Formats:

- .dat files with specific integer header formats
- .iqs files with proprietary OAI structures
- Custom binary formats designed for OAI compatibility

<https://gitlab.eurecom.fr/oai/openairinterface5g/-/tree/develop/radio/rfsimulator#store-and-replay>

Initially i ran into some format errors. The replay_node format is attached with this please refer to it.

To convert the iq samples from sigmf to oai compatible format we used a custom python script to get it into appropriate format. Which can be found here --->

[NR.py](#)

Refer to document 1-3.pdf for generation of iq samples and storing them.

We were running into a timing alignment issue and we reversing the roles of UE and replay_node. Initially the replay_node was acting as a client and UE works as the server we tried reversing it. It fixes the time alignment issue yet no demodulation occurs which is strange.

OpenAirInterface5G Function Reference

Through my work on this project, I've mapped out the complete structure of OpenAirInterface5G's 5G NR implementation and identified where each critical function lives within the codebase. What started as a daunting exploration through thousands of files turned into a clear understanding of how OAI organizes its signal processing pipeline. The key breakthrough was discovering that all the essential functions - from initial cell synchronization with `nr_initial_sync()` to polar decoding with `polar_decoder()` - follow a logical directory structure that mirrors the 5G protocol stack itself.

The most valuable insight was realizing that OAI separates concerns beautifully: physical layer functions live in `openair1/PHY/`, transport mechanisms are in `NR_UE_TRANSPORT/`, while coding algorithms have their own dedicated spaces in `CODING/`. This organization makes it much easier to extract specific functions for integration into GNU Radio blocks.

Please refer to `filepaths.pdf` for more information about each function and its usage.

Based on the `WORKFLOW.pdf` document, let me break down the comprehensive GNU Radio implementation strategy that was developed:

The GNU Radio Workflow Architecture

The workflow document outlines a systematic six-stage approach to implementing a 5G cell scanner in GNU Radio, carefully balancing native GNU Radio capabilities with custom 5G-specific processing blocks.

Stage 1: Time & Frequency Synchronization (PSS/SSS Search)

This foundational stage establishes the critical timing and frequency reference needed for all subsequent processing:

Block A - PSS/SSS Sequence Generation: Uses custom blocks to generate the synchronization sequences. This was necessary because the PSS and SSS sequences follow 5G NR-specific generation algorithms that aren't available in standard GNU Radio.

Block B - FFT of Received Samples: Leverages GNU Radio's native FFT block, one of its most optimized components. This demonstrates the strategy of using native blocks wherever possible.

Block C - PSS Correlation: Implements correlation using GNU Radio native blocks through multiply conjugate + IFFT operations, or alternatively using the correlation estimator. The workflow shows flexibility in implementation approaches.

Block D - SSS Correlation: Similar approach to PSS correlation, maintaining consistency in the correlation methodology.

The stage includes decision logic - if PSS peak isn't detected, the system continues scanning. Once both PSS and SSS are successfully correlated, Block E combines NID2 from PSS with NID1 from SSS to resolve the full Physical Cell ID, followed by frequency offset computation and refinement using PLL/Costas loop techniques.

Stage 2: PBCH Extraction + Demodulation

This stage focuses on extracting and demodulating the Physical Broadcast Channel to decode the Master Information Block:

Block F - FFT of PBCH Symbols: Again uses GNU Radio's native FFT implementation for consistency and performance.

Block G - PBCH Resource Element Extraction: Requires custom implementation since PBCH resource element mapping follows 5G NR-specific patterns not available in standard GNU Radio.

Block H - Channel Estimation from PBCH DMRS: Implements channel estimation using PBCH Demodulation Reference Signals. The workflow suggests both custom implementation and native approximation options - Simple LS estimation can be approximated using Multiply Const, Complex to Float, and Complex to Mag blocks, while robust MMSE requires custom implementation.

Block I - Channel Equalization: Uses the channel estimates to equalize the received signals.

Block J - QPSK Demodulation: Leverages native GNU Radio blocks (Complex to Float + QPSK Demod) for standard modulation schemes.

Stage 3: PBCH Polar Decoding

Block K - Polar Decoding: Requires custom implementation since polar codes are 5G NR-specific and not available in standard GNU Radio libraries.

Block L - ASN.1 MIB Parsing: Custom block for parsing the ASN.1-encoded Master Information Block to extract critical system information.

Stages 4-6: Progressive Decoding Chain

The workflow continues through increasingly complex processing:

Stage 4: CORESET Detection and Configuration

Stage 5: PDCCH Processing and DCI Decoding

Stage 6: SIB1 Processing with LDPC Decoding and ASN.1 Parsing

Each stage builds upon the previous ones, creating a complete signal processing pipeline from initial synchronization through full system information decoding.

Refer to WORKFLOW.pdf.

Take a look at pss_nr.py which implements openairinterface5g's logic in a python block.